

PRACTICAL EXAMINATION REPORT

LIBRANOVA

"Your Books. Your Knowledge. Your Library."

Roll Number: 24DCS074

Batch: 5CSE1-C

Exam Set: Set B

Course Code: ITUE301

Repository: itue301-exam-24dcs074-5CSE1-C

Date: August 20, 2026

Component	Technology / Implementation
Frontend Framework	React 18 + React Router v6 + Vanilla CSS
Backend REST API	Node.js + Express.js + Custom Middleware
Database & ODM	MongoDB + Mongoose Schema Validation
Theme & Aesthetics	Classical Academia (Mahogany, Parchment, Brass Gold)

2. Project Overview

LIBRANOVA is a complete, full-stack Library Book Management System built from scratch for the Set B Practical Examination. The application embodies a prestigious classical Academia aesthetic inspired by historical scholarly libraries (featuring dark mahogany tones, parchment elements, gold borders, and Roman volume headers). The system allows users to seamlessly explore book archives, view availability indicators, trigger quick book preview modals, filter available volumes, and submit controlled borrowing requisitions with real-time feedback.

3. Technology Stack

Layer	Technology	Role / Function
Frontend	React 18 & React Router v6	Single Page Application routing, component state, & dynamic rendering
Styling	Vanilla CSS + Google Fonts	Academia theme design system (Cinzel, Cormorant Garamond)
Backend	Node.js & Express.js	RESTful API routes, logging, and global error handling
Database	MongoDB & Mongoose	NoSQL storage, schemas, references, and validation constraints
HTTP Client	Native Fetch API	Asynchronous API requests with loading/error handling

4. Task 1–5 Implementation Details

Task 1 — React Components: Implemented five core components: HomePage, BooksPage, BorrowPage, BookCard, and Navigation. The BookCard component receives props (title, author, category, available) and displays visually distinct badges for Available (Emerald green badge) vs Not Available (Crimson red badge).

Task 2 — Routing & State: Configured client-side routing using React Router v6 (<BrowserRouter>, <Routes>, <Route>) with <Link> elements navigating between /, /books, and /borrow without triggering full browser reloads. Implemented controlled form inputs in BorrowPage (Member Name, Book Title, Borrow Date, Return Date) using useState with a live updating Parchment Summary card.

Task 3 — Express REST API & Middleware: Developed Node/Express backend with in-memory API endpoints: GET /api/v1/books (status 200), GET /api/v1/borrowings (status 200), and POST /api/v1/borrowings (status 201). Created global requestLogger middleware logging [METHOD] [PATH] [TIMESTAMP] and global error-handling middleware placed as the last middleware to return structured JSON errors without exposing raw stack traces.

Task 4 — React API Consumption: BooksPage consumes the Express API using useEffect() and fetch('http://localhost:5000/api/v1/books'). Maintained data, loading, and error states with dedicated UI components for loading spinners, error retry alerts, and success grid rendering via BookCard.

Task 5 — MongoDB & Mongoose Integration: Defined three Mongoose models in models/: Book.js (required title/author/category, unique ISBN, default available true), Member.js (required name/email/dept, unique email), and Borrowing.js (references to Member and Book, borrow/return dates, status enum restricted to ['borrowed', 'returned',

'overdue'])). Connected via MONGO_URI from .env and demonstrated validation error handling returning 400 Bad Request.

5. Interactive UX Feature & How to Run

Quick Book Preview & Available Only Filter: Clicking any BookCard triggers an ornate modal showing title, author, category, availability, and description. An **Available Only** toggle button allows instant state filtering on fetched API data using useState without extra backend requests.

How to Run Application:

1. Start Backend: `cd backend && npm install && node server.js` (Port 5000)
2. Start Frontend: `cd frontend && npm install && npm run dev` (Port 5173)
3. Access Browser: Open `http://localhost:5173`

6. Viva Key Concepts Reference

Concept	Key Explanation for Viva
Components & Props	Components are reusable UI functions returning JSX. Props are read-only inputs passed from parent to child components.
React Router & Link	Enables client-side SPA routing without full page reload. Link updates URL history and renders matched route components.
Controlled Inputs	Form inputs bound to React state via value and onChange, making React the single source of truth.
useEffect & fetch	useEffect performs side effects after render (fetching API data on mount). fetch returns Promises for async HTTP calls.
Middleware & Error Handling	Functions executing in request-response cycle. Error handler (4 args) placed last formats structured JSON error responses.
Mongoose Validation & Enum	Schemas enforce types and constraints (required, unique, enum). Invalid data triggers ValidationError caught by Express.

7. Implementation Evidence & Screenshots

Note: In accordance with examination guidelines, structured empty placeholder boxes are provided below for inserting evaluation screenshots.



Screenshot 1 — React Application: Demonstrating LIBRANOVA classical UI, navigation header, book cards, and live borrow form state.

[SCREENSHOT PLACEHOLDER 2]
Express REST API Request & Response (Postman / Browser GET /api/v1/books)

Screenshot 2 — REST API/Postman: Demonstrating GET /api/v1/books response (200 OK) and requestLogger terminal logs.

7. Implementation Evidence (Continued)

[SCREENSHOT PLACEHOLDER 3]
MongoDB Compass / Atlas Collection & Mongoose Validation Failure

Screenshot 3 — MongoDB Compass/Atlas: Demonstrating MongoDB database connection, Mongoose model documents, and validation error responses.